VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY

Experiment 9
Mini Project Report

APPLICATION OF NATURE-INSPIRED COMPUTING TO A REAL-WORLD OPTIMIZATION PROBLEM

EXAMINATION TIMETABLE SCHEDULING USING PARTICLE SWARM OPTIMIZATION

STUDENT	Hardik Hinduja
ROLL NUMBER	20
CLASS	D16 AD A/B
SUBJECT	NIA Lab
PROJECT TEAM	Hardik Hinduja 20 | Avinash Gehi 11 | Sahil Deshmukh 9 | Siddharth Dey 10

Department of Artificial Intelligence and Data Science
Vivekanand Education Society's Institute of Technology
Experiment 9 | Submitted November 2026

This report documents a reproducible synthetic benchmark, a discrete PSO scheduler, an independent clash audit, and the project website. It follows the supplied mini-project brief and identifies the limits of the generated data.

1. Project Summary and Problem Formulation

This project assigns each examination to a period while reducing simultaneous exams taken by the same student. A weighted conflict graph records how many students take each exam pair. The primary objective is to minimize the student clash count. When clash scores tie, the schedule uses fewer periods and balances the load.

The selected assignment topic is Project 2, Examination Timetable Scheduling Using PSO. It is a strong mini-project choice because the optimization objective is observable, the result can be audited directly against registrations, and the timetable has a clear interactive representation.

Study design

Item	Specification
Synthetic students	120, arranged in three cohorts of 40
Exams	12, four per synthetic cohort
Available periods	8 periods over four days
Student course load	3 or 4 exams from the student's cohort
PSO parameters	32 particles; 100 iterations; inertia 0.68; c1 = c2 = 1.42
Reproducibility	Instance seed 42; search seed 141
Primary metric	Weighted simultaneous exam pairs; independent student-pair audit
Secondary metrics	Periods used, load spread, runtime

Decision variables and constraints

The decision variable xi in {0,...,7} gives the period for exam i. Every exam is assigned once, all period indexes must remain within the horizon, and no student should have two exams in the same period. The decoder prioritizes conflict-heavy exams and repairs candidate allocations before fitness comparison. Room capacity, invigilator availability, course precedence, variable exam lengths, and accessibility are outside this prototype.

Privacy and use guardrails

The notebook creates anonymous synthetic registrations only. It does not request student identifiers or grades. After optimization, assertions check exam count, period bounds, and agreement between the weighted objective and a separate audit of every student's exam pairs. A zero-clash outcome is reported only when the independent audit is zero. Results are illustrative and are not ready for institutional use.

2. Method and Algorithm

A particle stores one real-valued period preference per exam. At each iteration, velocity combines inertia, attraction to that particle's personal best, and attraction to the swarm's global best. Positions are clipped to the period range and decoded into discrete schedules.

Fitness formula

```
fitness = clash_pairs * 10,000 + periods_used * 100 + load_spread
```

The large clash coefficient gives feasibility priority. Period count is optimized next; load spread breaks remaining ties. The conflict-aware decoder is deterministic for a given position and instance.

Pseudocode

```
initialize particle preferences and velocities
score each candidate after degree-ordered repair
for each iteration:
    update velocity toward personal and global best
    clip positions to the available period range
    repair each candidate to reduce weighted clashes
    update personal and global bests by fitness
independently audit the final assignment
export schedule, benchmark, and audit tables
```

Guardrail checks in the notebook

Check	Expected result	Observed result
Every exam appears once	12 unique exams	PASS: 12 unique exams
Each period is in bounds	0 through 7	PASS: all in range
Independent audit agrees with objective	equal counts	PASS: 0 = 0
Student clash pairs	0 for this generated instance	PASS: 0

Implementation excerpt from the Colab notebook

```
for iteration in range(iterations):
    for particle in swarm:
        for d in range(exams):
            r1, r2 = rng.random(), rng.random()
            velocity[d] = (0.68 * velocity[d]
                + 1.42 * r1 * (personal_best[d] - position[d])
                + 1.42 * r2 * (global_best[d] - position[d]))
            position[d] = clip(position[d] + velocity[d], 0, periods - 1)
        plan = decode_repair(position, conflict_weights, periods)
        score = evaluate(plan, conflict_weights, periods)
        update_personal_and_global_best(score)
```

The notebook includes the complete implementation. This excerpt shows the PSO movement and evaluation loop; candidate repair and objective code are also included there.

3. Benchmark Results and Timetable

All methods use the same synthetic registration instance. The random baseline assigns periods directly and is not repaired. Degree-greedy repair is a deterministic comparison. The PSO uses the same repair decoder and optimizes its particle preferences.

Benchmark comparison

Method	Clash pairs	Periods	Load spread	Fitness	Runtime (s)
Random assignment	51	7	0.700	510700.7	0.0001
Degree-greedy repair	0	4	0.000	400.0	0.0004
Discrete PSO	0	4	0.000	400.0	1.8725

On this generated instance, random assignment yields 51 student clash pairs. Both repaired methods produce zero clashes using four periods. PSO reaches the same feasible objective as the greedy baseline, so this small case does not show PSO outperforming the baseline. Runtime is machine-dependent; the figures above were measured for this report build.

Optimized timetable

Exam	Registrations	Day	Session	Period
EX-01	30	Tue, 10 Nov	13:00-16:00	4
EX-02	33	Wed, 11 Nov	09:00-12:00	5
EX-03	33	Wed, 11 Nov	13:00-16:00	6
EX-04	34	Tue, 10 Nov	09:00-12:00	3
EX-05	29	Tue, 10 Nov	13:00-16:00	4
EX-06	32	Wed, 11 Nov	13:00-16:00	6
EX-07	35	Tue, 10 Nov	09:00-12:00	3
EX-08	34	Wed, 11 Nov	09:00-12:00	5
EX-09	33	Wed, 11 Nov	09:00-12:00	5
EX-10	32	Wed, 11 Nov	13:00-16:00	6
EX-11	30	Tue, 10 Nov	13:00-16:00	4
EX-12	35	Tue, 10 Nov	09:00-12:00	3

Illustrative schedule labels are based on 9 November 2026. Actual exam dates and periods are demonstration metadata, not institutional bookings.

Period utilization

Period	Exams	Exam IDs	Student registrations
3	3	EX-04, EX-07, EX-12	104
4	3	EX-01, EX-05, EX-11	89
5	3	EX-02, EX-08, EX-09	100
6	3	EX-03, EX-06, EX-10	97

4. Conflict Graph and Data Tables

Each matrix cell counts the synthetic students who take both exams in that row and column. The diagonal is zero. Values outside each cohort's four-exam block are zero by design.

Conflict weight matrix

Exam	EX-01	EX-02	EX-03	EX-04	EX-05	EX-06	EX-07	EX-08	EX-09	EX-10	EX-11	EX-12
EX-01	0	23	23	24	0	0	0	0	0	0	0	0
EX-02	23	0	26	27	0	0	0	0	0	0	0	0
EX-03	23	26	0	27	0	0	0	0	0	0	0	0
EX-04	24	27	27	0	0	0	0	0	0	0	0	0
EX-05	0	0	0	0	0	21	24	23	0	0	0	0
EX-06	0	0	0	0	21	0	27	26	0	0	0	0
EX-07	0	0	0	0	24	27	0	29	0	0	0	0
EX-08	0	0	0	0	23	26	29	0	0	0	0	0
EX-09	0	0	0	0	0	0	0	0	0	25	23	28
EX-10	0	0	0	0	0	0	0	0	25	0	22	27
EX-11	0	0	0	0	0	0	0	0	23	22	0	25
EX-12	0	0	0	0	0	0	0	0	28	27	25	0

Anonymous registration sample

Synthetic ID	Exam list	Count
S-001	EX-01, EX-02, EX-03, EX-04	4
S-002	EX-01, EX-03, EX-04	3
S-003	EX-02, EX-03, EX-04	3
S-004	EX-01, EX-02, EX-03	3
S-005	EX-01, EX-02, EX-03, EX-04	4
S-006	EX-02, EX-03, EX-04	3
S-007	EX-01, EX-02, EX-03	3
S-008	EX-01, EX-02, EX-04	3
S-009	EX-01, EX-02, EX-03, EX-04	4
S-010	EX-01, EX-03, EX-04	3
S-011	EX-02, EX-03, EX-04	3
S-012	EX-02, EX-03, EX-04	3

Conflict objective and audit

Metric	Result
Total nonzero conflict weight across all exam pairs	450
Student clash pairs in PSO schedule	0
Independent registration-level audit	0
Consistency check	PASS: objective 0 = audit 0

5. Colab Notebook and Reproducibility

The linked Colab notebook is built around the blank notebook supplied with the project request. Its code bootstraps the public repository copy, runs all notebook cells, displays the tables and monochrome conflict plot, and writes CSV exports into the Colab runtime.

Google Colab notebook

https://colab.research.google.com/drive/1GxqpVK-q_ZGQ7XSdVVrjEuHLYpR0fctJ

Notebook output files

File	Contents
benchmark_results.csv	Baseline and PSO objective, periods, fitness, runtime, feasibility
optimized_timetable.csv	One period per exam and registration count
synthetic_registration_sample.csv	Anonymous synthetic student exam selections
conflict_weight_matrix.csv	Shared-student counts for every exam pair
guardrail_audit.csv	Post-run invariant and clash checks

Reproducibility procedure

Open the notebook link and run all cells. Seed 42 regenerates the same synthetic registrations; the search seed is 141. The objective values are deterministic for this implementation. Wall-clock timing is reported locally and can vary across Colab runtimes.

Monochrome code excerpt

```
def fitness(plan, conflict_weights, periods):
    clashes = conflict_count(plan, conflict_weights)
    used = len(set(plan))
    spread = load_standard_deviation(plan, periods)
    return clashes * 10_000 + used * 100 + spread

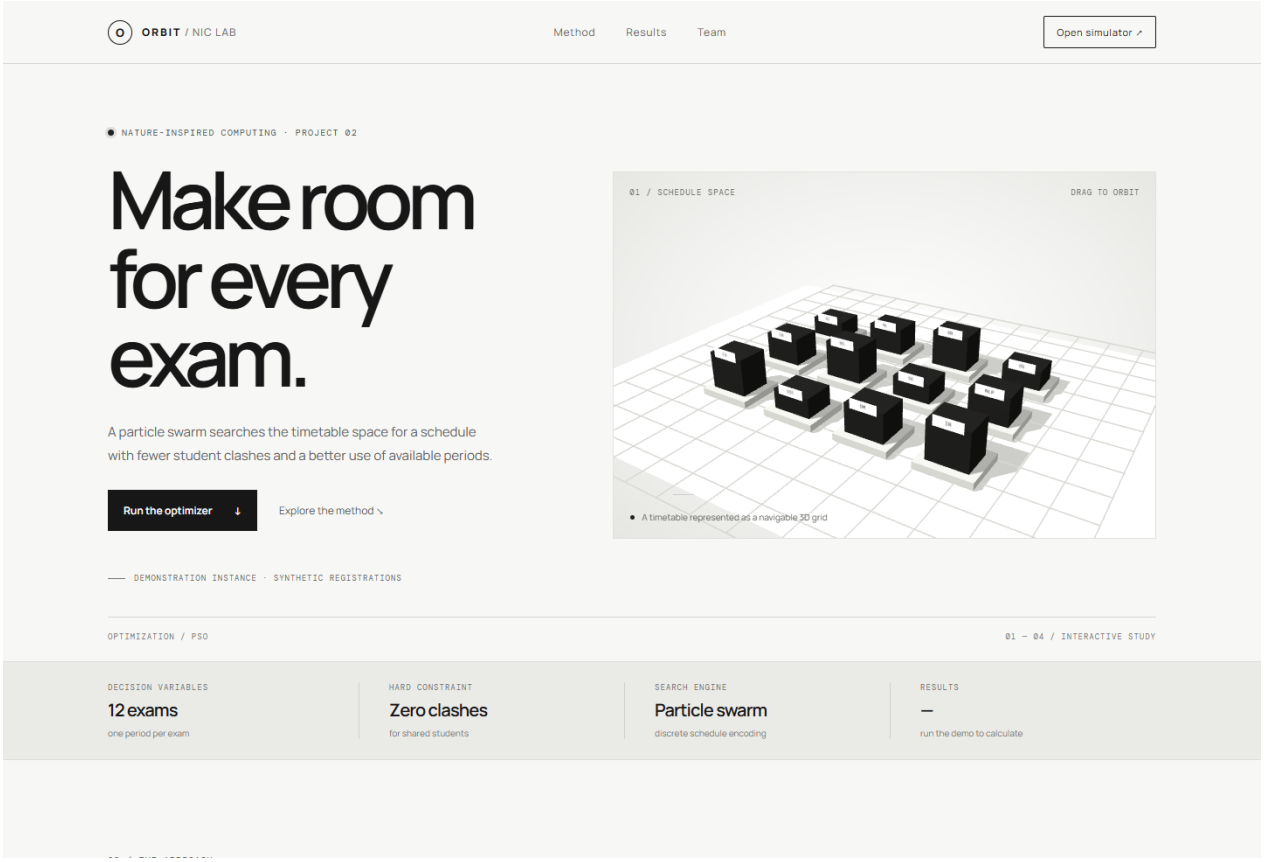
# Independent student-level audit
audit = sum(1 for student in registrations
    for i, left in enumerate(student)
    for right in student[i+1:]
    if schedule[left] == schedule[right])
assert audit == reported_clashes
```

Limitations

This prototype does not model room capacity, invigilators, student accessibility, course sequencing, exam duration, or real enrollment data. The search is heuristic. One feasible synthetic result is not evidence of global optimality. Any production schedule requires validation by the institution's data owner and academic administrators.

6. Website Evidence

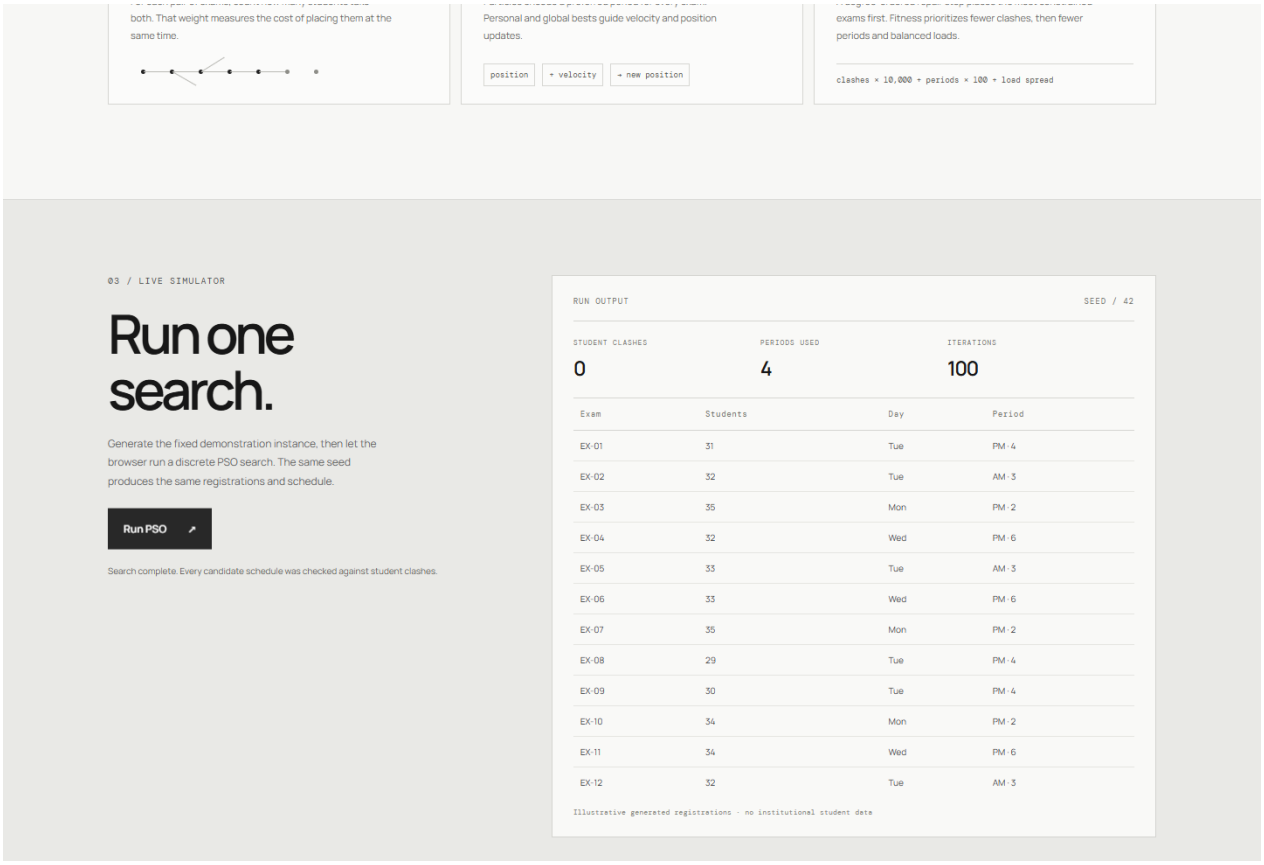
Website preview capture from the exact static build uploaded to Vercel: monochrome landing page and interactive 3D examination grid.



The page uses a white background, black text, grayscale 3D geometry, and a responsive layout. The 3D model can be rotated by dragging; the simulator runs the same seeded cohort instance and exposes its schedule. Vercel Drop reported a temporary live URL, but this browser received HTTP 403 when it attempted to open that host. The included images are the local preview of the exact static build that was uploaded.

Website Evidence Continued

Website preview capture from the exact static build uploaded to Vercel: PSO output with zero audited clashes across four periods.



References and Project Files

Supplied Experiment 9 Mini-Project brief: Application of Nature-Inspired Computing Algorithms for Solving Real-World Optimization Problems; Project 2: Examination Timetable Scheduling Using PSO.

Project source: https://github.com/Hardik182005/NIA
Temporary Vercel deployment (expires unless claimed): https://fleet-bayou-9rbyyz8.tmp.now

Implementation prompt: PROJECT_PROMPT.md. Full algorithm, baseline code, tables, audit assertions, plots, and CSV exports: notebooks/NIA_Exam_Timetable_PSO.ipynb. Website source and static deployment files: site/dist/.

Prepared for	NIA Lab | Experiment 9 Mini Project
Selected topic	Examination Timetable Scheduling Using PSO
Project team	Hardik Hinduja (20), Avinash Gehi (11), Sahil Deshmukh (9), Siddharth Dey (10)
Data notice	All registration records are generated and synthetic.